COLLEGE OF TECHNOLOGY AND BUILT ENVIRONMENT

SCHOOL OF INFORMATION TECHNOLOGY AND ENGINEERING

DEPARTMENT OF IT/SE ENG.

ModIntel - AI Enhanced ModSecurity

Software Requirements Specification

Team Members

Meaza Tadele	UGR/6378/14
Fozia Mohammed	UGR/4535/14
Liyu Desta	UGR/6694/13
Natanim Kemal	UGR/4648/14
Eyoab Amare	UGR/5756/14

Advisors: Mr. Nebiat

REVISION HISTORY

Date	Description	Author	Comments
Jan 2026	Version 1.0	Team	-

Advisor approval will be added separately upon review.

_____

Contents

DOCUMENT APPROVAL ..... **V**
LIST OF TABLES ..... **VI**
LIST OF FIGURES ..... **VII**
DEFINITIONS, ACRONYMS, AND ABBREVIATIONS ..... **VIII**
DECLARATION ..... **IX**
1. INTRODUCTION **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Overview 2
2. GENERAL DESCRIPTION **2**
1.1 Product Perspective 2
2.2 Product Functions 3
2.3 User Characteristics 3
2.4 General Constraints 3
2.5 Assumptions and Dependencies 4
3. SPECIFIC REQUIREMENTS **4**
3.1 External Interface Requirements 4
3.1.1 User Interfaces 4
3.1.2 Hardware Interfaces 6
3.1.3 Software Interfaces 7
3.1.4 Communications Interfaces 8
3.2 Functional Requirements 9
3.2.2 Feature #2: Analyst-Guided Alert Review and Prioritization 9
3.2.3 Feature #3: Offline Training and Evaluation of Detection Models 10
3.2.4 Feature #4: Controlled Model Deployment and Recovery 11
3.2.5 Feature #5: Security Logging and Decision Traceability 11
3.2.6 Feature #6: Administrative Control of Decision Policies 12
3.2.7 Feature #7: Explanation of Assisted Security Decisions 13
3.3 Use Cases 14
UC-01: ML-Assisted Request Classification 14
UC-02: Human-in-the-Loop Alert Prioritization 15

UC-03: Offline Model Training and Evaluation 16
UC-04: Safe Model Deployment and Rollback 17
UC-05: Audit Logging and Traceability 17
UC-06: Configurable Decision Policy Management 18
UC-07: ML Decision Explanation and Justification 18
3.4 Non-Functional Requirements 20
3.4.1 Performance 20
3.4.2 Reliability 20
3.4.3 Availability 20
3.4.4 Security 20
3.4.5 Maintainability 21
3.4.6 Portability 21
3.5 Inverse Requirements 21
3.6 Design Constraints 22
3.7 Logical Database Requirements 23
3.8 Other Requirements 24
4. CHANGE MANAGEMENT PROCESS 25
REFERENCES 26
APPENDICES 27
Appendix 1 27

DOCUMENT APPROVAL

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
		Instructor	
			January 24, 2026

LIST OF TABLES

Table	Page
Table 1: Use Case Summary	14
Table 2: Use Case Traceability Matrix	19

LIST OF FIGURES

Figure	Page
Figure 1: UML Use Case Diagram	19

DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

Abbreviation	Stands for
CRS	OWASP Core Rule Set
WAF	Web Application Firewall
ML	Machine Learning
FP	False Positive
TP	True Positive
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
IPC	Inter-Process Communication

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Group Members: Meaza Tadele, Fozia Mohammed, Liyu Desta, Natanim Kemal, Eyoab Amare

Date: January 2026

1. Introduction

This Software Requirements Specification (SRS) document defines the requirements for Modintel, a hybrid Web Application Firewall (WAF) solution that integrates Machine Learning techniques with the traditional rule-based ModSecurity engine. The purpose of this document is to provide a complete, precise, and verifiable description of the system requirements to guide system designers, developers, testers, and stakeholders throughout the development lifecycle.

The SRS serves as the primary reference for understanding what the system is expected to do, the constraints under which it operates, and the non-functional requirements it must satisfy. It ensures a shared understanding between project stakeholders and provides a foundation for system design, implementation, and validation.

1.1 Purpose

The purpose of this SRS is to formally specify the functional and non-functional requirements of Modintel (AI-enhanced Modsecurity), which aims to reduce false positive alerts generated by the OWASP Core Rule Set (CRS) without compromising web application security.

This document is intended for software engineers, system architects, security analysts, testers, academic evaluators, and future maintainers who require a clear and consistent understanding of the system requirements.

1.2 Scope

The software product described in this document is named Modintel. It is designed to operate as an intelligent extension to the existing ModSecurity framework and the OWASP Core Rule Set.

The system will:

- Monitor HTTP requests processed by ModSecurity,
- Collect and analyze audit logs, extract security-relevant features,

- Apply a Machine Learning–based decision support classifier to identify likely false positive alerts,
- Assist security administrators by prioritizing alerts and reducing alert fatigue.

The system will not:

- Replace ModSecurity
- Automatically override security decisions
- Provide enterprise-grade deployment features.

1.3 Overview

This SRS document is organized into four main sections. Section 1 introduces the document and defines its purpose and scope. Section 2 provides a general description of the system context and user characteristics. Section 3 details the specific functional and non-functional requirements. Section 4 describes the change management process, followed by references and appendices.

2. General Description

This section provides a general overview of Modintel. It describes the system context, major functions, user characteristics, constraints, assumptions, and dependencies. This section does not define specific requirements but offers background information to help understand the detailed requirements presented later

1.1 Product Perspective

Modintel is designed as an extension to the existing ModSecurity Web Application Firewall framework integrated with the OWASP Core Rule Set (CRS). It does not operate as a standalone security solution, but rather as a complementary decision-support layer that enhances ModSecurity's alert handling capabilities.

The system processes ModSecurity audit logs generated during HTTP request inspection and applies Machine Learning techniques to analyze alert patterns. The ML component

functions as a secondary opinion mechanism, assisting administrators in identifying likely false positive alerts without altering the core ModSecurity detection logic.

2.2 Product Functions

At a high level, the Modintel performs the following functions:-

- Collects ModSecurity audit logs generated during web traffic inspection
- Extracts relevant security features such as rule IDs, anomaly scores, and request metadata
- Applies a trained Machine Learning classifier to evaluate flagged requests
- Generates confidence scores indicating the likelihood of false positive alerts and displays it on dashboard
- Prioritizes alerts to assist security administrators in manual review
- Supports evaluation of system effectiveness using security performance metrics

2.3 User Characteristics

The intended users of Modintel include:

Security Administrators – Users responsible for managing ModSecurity configurations and reviewing alerts. They are expected to have a basic understanding of WAF concepts and HTTP traffic.

Security Analysts and Researchers – Users analyzing alert behavior and evaluating the effectiveness of AI-assisted intrusion detection techniques.

System Developers and Maintainers – Users responsible for maintaining and extending the system. They are expected to have knowledge of Python, ModSecurity, and basic Machine Learning concepts.

2.4 General Constraints

The following constraints limit the design and implementation options of the system:

- The system shall operate as an extension to an existing ModSecurity deployment and shall not replace or modify its core rule-based engine.
- The system shall comply with organizational security policies that prohibit automated blocking decisions based solely on machine learning output.

- The system shall function within the resource limitations of a standard server environment used for web application deployment.
- The system shall support auditability, ensuring all decisions and system actions can be reviewed after execution.
- The system is considered a security-critical application, requiring conservative failure handling and predictable behavior.
- The system shall adhere to safe software engineering practices suitable for security-sensitive environments.

2.5 Assumptions and Dependencies

The system requirements are based on the following assumptions and dependencies:

- It is assumed that ModSecurity is correctly installed and configured in the target deployment environment.
- It is assumed that HTTP request and audit logs are accessible for feature extraction and offline analysis.
- The system assumes the availability of security analysts or administrators to review alerts and provide feedback.
- It is assumed that the system will be deployed in a controlled academic or experimental environment.
- Changes to the ModSecurity rule set or deployment architecture may require corresponding updates to the system requirements.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system provides multiple user interfaces tailored to different user roles involved in web application security operations. These roles include security administrators, security analysts, and system operators. The interfaces are designed to support both real-time

decision-making and offline analysis, while minimizing cognitive overload caused by excessive security alerts.s

Logical Characteristics of User Interfaces

1. Web-Based Admin Dashboard
    - Provides visualization of ModSecurity alerts, ML classification results, and anomaly scores.
    - Displays request metadata (timestamp, URI, HTTP method, triggered CRS rule IDs, anomaly score).
    - Supports filtering and sorting by:

      It provides filtering and sorting mechanisms that allow users to organize security alerts based on prediction confidence scores generated by the machine learning models, triggered ModSecurity rule identifiers, detected or inferred attack categories, and the estimated likelihood that an alert represents a false positive. These capabilities enable analysts to quickly prioritize critical events while suppressing low-risk noise.
    - Enables security analysts to mark alerts as:
        - True attack
        - False positive
        - Requires further investigation
    - Supports historical comparison

      Historical comparison views that allow users to examine and contrast decisions produced by vanilla ModSecurity rule-based detection with those generated using machine-learning-assisted analysis. This comparison capability adds or enhances transparency, supports auditing and compliance activities, and enables informed assessment of the effectiveness of the ML-enhanced approach over time.
2. Model Monitoring Interface
    - Shows ML model performance metrics such as false-positive rate, recall, precision, and inference latency.
    - Displays drift indicators when traffic characteristics deviate from training data.
3. Command-Line Interface (CLI)
    - Used by system operators for:

- Deploying model
- Enabling or disabling ML inference
- Switching ModSecurity operational modes (DetectionOnly / Blocking)

- Suitable for automation and CI/CD pipelines.

Interface Optimization Considerations

The user interfaces are designed to reduce alert fatigue by prioritizing alerts that are likely to be false positives, in alignment with the human-in-the-loop workflow adopted by the system. The dashboard deliberately avoids exposing raw machine learning internals and instead presents interpretable security-oriented outputs, such as rule relevance indicators and prediction confidence scores, to support informed decision-making. Accessibility principles, including adequate visual contrast, readable typography, and keyboard navigation support, are incorporated to ensure usability for a broad range of users, including students and security trainees. The system makes no assumption that users possess machine learning expertise, and all explanations and visualizations are framed in security and operational terms rather than mathematical abstractions.

3.1.2 Hardware Interfaces

The system does not directly control or manipulate physical hardware devices. It operates entirely at the software and network level.

Hardware Interaction Characteristics

The system executes on standard server infrastructure, whether deployed on physical machines or virtualized environments, and does not require any specialized hardware components. It passively inspects HTTP traffic that is already processed by ModSecurity, operating entirely at the software level without direct interaction with physical devices. While the underlying machine learning libraries may optionally leverage hardware-level optimizations such as CPU vectorization, such acceleration is fully abstracted from the system design and imposes no explicit hardware interface requirements.

Statement of Applicability

The system has no explicit hardware interface requirements.

All hardware dependencies are abstracted by the operating system and runtime environment.

3.1.3 Software Interfaces

This system must interface with several pre-existing, externally defined software components, which are not design choices but functional necessities dictated by the problem domain.

Required Software Products

1. ModSecurity Web Application Firewall

a) Name: ModSecurity
b) Mnemonic: MODSEC
c) Specification: OWASP ModSecurity Reference Manual
d) Version: v3.x
e) Source: OWASP Foundation

Acts as the primary inspection engine for HTTP requests and executes the OWASP Core Rule Set. The system extracts rule-trigger data from ModSecurity audit logs to generate ML features.

Interface Definition:

- Input: HTTP request data
- Output: Audit logs containing:
  - Triggered rule IDs
  - Anomaly scores
  - Request metadata
- Format: Structured text or JSON audit logs

2. OWASP Core Rule Set (CRS)

a) Name: OWASP CRS
b) Mnemonic: CRS
c) Specification: OWASP CRS Documentation
d) Version: v4.x
e) Source: OWASP Foundation

Provides the rule-based detection baseline. CRS rule activations are treated as binary or count-based features for ML models rather than final decision logic.

3. Machine Learning Runtime

a) Name: Python ML Stack
b) Mnemonic: MLRT
c) Specification: scikit-learn API
d) Version: 1.x
e) Source: scikit-learn Project

Executes trained ML models (Logistic Regression, SVM, Decision Tree, Random Forest) during both offline evaluation and online inference phases.

Interface Definition:

- Input: Feature vectors derived from CRS rule activations
- Output: Classification label and confidence score
- Format: In-memory structured data (arrays / vectors)

3.1.4 Communications Interfaces

Network Protocols

1. **HTTP / HTTPS**
   - Used for inbound client traffic inspected by ModSecurity.
   - The system must correctly parse requests without modifying transport semantics.
2. **Local Inter-Process Communication (IPC)**
   - Used between ModSecurity and the ML inference component.
   - Implemented via:
     - Unix sockets
     - Shared memory
     - Local REST endpoint
   - Latency requirements are strict to avoid degrading request throughput.
3. **File-Based Log Transport**
   - ModSecurity audit logs are consumed by offline training components.
   - Log format must remain consistent across versions to ensure reproducibility.

No custom network protocol is exposed externally by the system.

3.2 Functional Requirements

3.2.1 Feature #1: Request Evaluation Using Machine Learning Support

The system shall analyze ModSecurity-generated audit data and provide machine-learning-based classification to assist in identifying potentially malicious HTTP requests.

3.2.1.1 Inputs

- ModSecurity audit logs
- Triggered rule identifiers
- Anomaly scores
- HTTP request metadata

3.2.1.2 Processing

The system shall extract relevant features from the input data and apply a trained machine learning model to assess the likelihood of malicious behavior.

3.2.1.3 Outputs

- Classification result indicating likelihood of maliciousness
- Confidence score associated with the classification

3.2.1.4 Error Handling

If classification fails or input data is invalid, the system shall record the error and proceed without ML assistance.

3.2.2 Feature #2: Analyst-Guided Alert Review and Prioritization

This feature ensures that ML outputs assist human analysts rather than replacing them, aligning with security best practices and reducing operational risk.

3.2.2.1 Inputs

- ML confidence scores
- ModSecurity alert logs
- Analyst feedback labels

3.2.2.2 Processing

1. Alerts predicted as likely false positives are prioritized.
2. Analysts review prioritized alerts through the dashboard.
3. Feedback is stored for future retraining.
4. Periodic offline retraining incorporates validated labels.

3.2.2.3 Outputs

- Updated alert status
- Reduced analyst workload
- Improved future model performance

3.2.2.4 Error Handling

- Analyst feedback conflicts are logged and flagged.
- Missing feedback does not interrupt normal operations.
- Model retraining is blocked if data integrity checks fail.

3.2.3 Feature #3: Offline Training and Evaluation of Detection Models

This feature enables training, validation, and evaluation of ML models using historical ModSecurity audit data in an offline environment. It Ensures thorough testing before deployment to live traffic inspection and minimizes operational risk

3.2.3.2 Inputs

- Archived ModSecurity audit logs
- Extracted rule activation data
- Labeled samples from analyst feedback
- Predefined training configurations (algorithms, feature encodings)

3.2.3.3 Processing

1. Transforms rule activations + request metadata into structured feature vectors
2. Trains and evaluates multiple ML models on consistent datasets
3. Computes performance metrics: false-positive rate, detection accuracy, precision-recall balance

3.2.3.4 Outputs

- Trained model artifacts

- Evaluation reports with performance metrics
- Validation results indicating deployment suitability

3.2.3.5 Error Handling

- Halts training on incomplete, corrupted, or insufficiently labeled data
- Reports issues clearly
- Produces no model artifacts unless all validation checks pass

3.2.4 Feature #4: Controlled Model Deployment and Recovery

This feature enables controlled and reversible deployment of ML models into live ModSecurity pipeline and preserves system stability and security guarantees

3.2.4.2 Inputs

- Validated model artifacts
- Deployment configuration parameters
- Operational mode settings (how ML affects decisions)

3.2.4.3 Processing

1. Loads model into inference runtime
2. Integrates with ModSecurity request evaluation flow
3. Deploys without traffic interruption
4. Monitors inference latency and classification consistency
5. Provides fallback to original rule-based mode on anomalies

3.2.4.4 Outputs

- Active ML-assisted decision path
- Deployment status logs
- Operational metrics (inference performance, system impact)

3.2.4.5 Error Handling

- Immediate revert to vanilla ModSecurity on deployment failure or runtime errors
- Full logging of all failures
- Automatic rollback of partial deployments

3.2.5 Feature #5: Security Logging and Decision Traceability

This feature provides comprehensive logging and traceability of security decisions and supports auditing, compliance, and forensic analysis.

3.2.5.2 Inputs

- ModSecurity alerts
- ML classification results
- Analyst feedback actions
- System configuration changes

3.2.5.3 Processing

1. Correlates rule-based detections, ML predictions, and human decisions
2. Creates unified audit records
3. Maintains full traceability: HTTP request → rules → ML outputs → analyst actions

3.2.5.4 Outputs

- Structured, queryable audit logs
- Usable for compliance reviews, investigations, performance assessment

3.2.5.5 Error Handling

- Buffers records locally during storage unavailability
- Resumes persistence when connectivity restored
- Logging failures never impact request processing or security enforcement

3.2.6 Feature #6: Administrative Control of Decision Policies

Allows administrators to control how ML outputs affect ModSecurity enforcement. It provides flexibility for different risk environments

3.2.6.2 Inputs

- Administrator-defined policy thresholds
- Confidence score cutoffs
- Operational modes (advisory vs enforcement)

3.2.6.3 Processing

1. Evaluates ML predictions against active policy
2. Decides: block / flag for review / allow
3. Supports policy updates without model retraining
4. Enables fast adaptation to new threats

3.2.6.4 Outputs

- Updated live decision behavior
- Configuration change logs (for audit)

3.2.6.5 Error Handling

- Rejects invalid or conflicting policy configurations
- Falls back to last known valid policy
- Continues normal operation during rejection

3.2.7 Feature #7: Explanation of Assisted Security Decisions

This feature provides transparent, human-readable explanations for ML-assisted decisions, enabling analysts to understand *why* a request was classified as malicious or a likely false positive. It strengthens analyst trust, supports forensic analysis, and aligns with security best practices requiring explainability in automated decision-support systems.

3.2.7.1 Inputs

- ML classification results
- Feature vectors derived from CRS rule activations
- Triggered CRS rule metadata
- Anomaly score values
- Active decision policy thresholds

3.2.7.2 Processing

1. Identifies the most influential features contributing to the ML classification result.
2. Correlates influential features with triggered CRS rules and request attributes.
3. Generates a structured explanation describing:
    a. Key rules influencing the decision
    b. Relative contribution of anomaly scores
    c. Confidence score interpretation against policy thresholds
4. Associates the explanation with the corresponding alert and audit record.

3.2.7.3 Outputs

- Human-readable decision explanation
- Structured explanation data linked to alerts
- Analyst-visible justification displayed via the dashboard

3.2.7.4 Error Handling

- If explanation generation fails, the system records the ML result without explanation.
- Explanation failures do not affect request handling or enforcement decisions.
- All failures are logged for diagnostic and audit purposes.

3.3 Use Cases

This section describes the interaction between system actors and Modintel system. Each use case is traceable to the functional requirements defined in Section 3.2. The use cases specify system behavior under normal and exceptional conditions and define how system components collaborate to achieve operational goals.

3.3.1 Actors

Primary Actors

- **ModSecurity Engine** – Performs rule-based inspection of HTTP requests.
- **Security Analyst** – Reviews alerts and provides feedback.
- **System Administrator** – Configures policies and manages model deployment.
- **ML Inference Module** – Performs machine learning classification.
- **Audit Logging System** – Stores security and decision records.

Secondary Actors

- **Web Client** – Sends HTTP requests to the protected application.
- **Explanation Engine** – Generates human-readable decision explanations.

3.3.2 Use Case Summary

Use Case ID	Use Case Name	Primary Actor	Related Functional Requirement
UC-01	ML-Assisted Request Classification	ModSecurity Engine	FR-3.2.1
UC-02	Human-in-the-Loop Alert Prioritization	Security Analyst	FR-3.2.2
UC-03	Offline Model Training and Evaluation	System Administrator	FR-3.2.3
UC-04	Safe Model Deployment and Rollback	System Administrator	FR-3.2.4
UC-05	Audit Logging and Traceability	Audit Logging System	FR-3.2.5
UC-06	Configurable Decision Policy Management	System Administrator	FR-3.2.6
UC-07	ML Decision Explanation and Justification	Security Analyst	FR-3.2.7

UC-01: ML-Assisted Request Classification

Primary Actor: ModSecurity Engine
Supporting Actor: ML Inference Module

Goal: Classify inspected HTTP requests as likely attacks or likely false positives using machine learning assistance.

Preconditions:

- ModSecurity with OWASP CRS is active.
- A trained ML model is available for inference.

Main Flow:

1. A web client sends an HTTP request to the protected application.
2. ModSecurity inspects the request using CRS rules.
3. Triggered rule IDs, anomaly scores, and request metadata are extracted.
4. The extracted data is transformed into a feature vector.
5. The ML Inference Module evaluates the feature vector.
6. The ML model returns a classification label and confidence score.
7. The system records the result together with the ModSecurity alert log.

Postconditions:

- The request is assigned a classification label and confidence score.
- A corresponding audit record is created.

Alternative Flow:

- If ML inference fails, the system defaults to standard ModSecurity behavior and logs the error.

UC-02: Human-in-the-Loop Alert Prioritization

Primary Actor: Security Analyst
Supporting Actor: Alert Dashboard

Goal: Enable analysts to review and prioritize alerts using ML confidence scores.

Preconditions:

- ModSecurity alerts have been generated.
- ML confidence scores are available.
- Analyst has access to the dashboard.

Main Flow:

1. The system displays alerts with associated ML confidence scores.

2. Alerts predicted as likely false positives are prioritized.
3. The analyst reviews selected alerts.
4. The analyst labels alerts as true attack or false positive.
5. Analyst feedback is stored for future model retraining.

Postconditions:

- Alert status is updated.
- Feedback data is saved.

Alternative Flow:

- If analyst feedback is missing, the system continues normal operations without interruption.

UC-03: Offline Model Training and Evaluation

Primary Actor: System Administrator

Goal: Train and validate machine learning models using historical audit logs.

Preconditions:

- Archived ModSecurity audit logs are available.
- Labeled datasets exist for training.

Main Flow:

1. The system loads historical audit logs.
2. Rule activation data and request metadata are extracted.
3. Feature vectors are constructed.
4. ML models are trained on labeled datasets.
5. Model performance is evaluated using defined metrics.
6. The validated model artifact is saved for deployment.

Postconditions:

- Trained model artifacts are produced.
- Evaluation reports are generated.

Alternative Flow:

- If training data is incomplete or corrupted, training is halted and an error report is produced.

UC-04: Safe Model Deployment and Rollback

Primary Actor: System Administrator

Goal: Deploy a validated ML model into the live ModSecurity environment safely.

Preconditions:

- A validated ML model artifact exists.
- Deployment configuration parameters are defined.

Main Flow:

1. The administrator initiates model deployment.
2. The system loads the ML model into the inference runtime.
3. The model is integrated into the ModSecurity request evaluation pipeline.
4. The system monitors inference latency and consistency.
5. ML-assisted decision support becomes active.

Postconditions:

- The ML-assisted decision path is operational.
- Deployment logs are created.

Alternative Flow:

- If deployment fails or runtime errors occur, the system reverts automatically to standard ModSecurity mode and logs the failure.

UC-05: Audit Logging and Traceability

Primary Actor: Audit Logging System

Goal: Maintain complete traceability of security decisions.

Preconditions:

- ModSecurity alerts and ML results are available.

Main Flow:

1. The system collects ModSecurity alerts.
2. ML classification results are attached to corresponding alerts.
3. Analyst feedback and policy decisions are linked to alert records.

4. Unified audit records are stored in structured logs.

Postconditions:

- Full traceability from HTTP request to final decision is preserved.

Alternative Flow:

- If log storage is unavailable, records are buffered locally and persisted later.

UC-06: Configurable Decision Policy Management

Primary Actor: System Administrator

Goal: Define how ML confidence scores influence enforcement decisions.

Preconditions:

- ML-assisted classification is active.
- Policy configuration interface is available.

Main Flow:

1. The administrator defines confidence score thresholds and operational mode.
2. The system validates the policy configuration.
3. The policy is applied to the live decision logic.
4. Requests are processed according to the updated policy.

Postconditions:

- Updated decision policies are active.
- Configuration changes are logged.

Alternative Flow:

- If an invalid policy is submitted, the system rejects it and reverts to the last valid policy.

UC-07: ML Decision Explanation and Justification

Primary Actor: Security Analyst
Supporting Actor: Explanation Engine

Goal: Provide human-readable explanations for ML-assisted decisions.

Preconditions:

- An ML classification result exists for an alert.

Main Flow:

1. The system identifies influential features contributing to the ML decision.
2. Influential features are correlated with triggered CRS rules.
3. A structured explanation is generated.
4. The explanation is displayed in the analyst dashboard and stored in audit logs.

Postconditions:

- Each ML-assisted alert has an associated explanation.

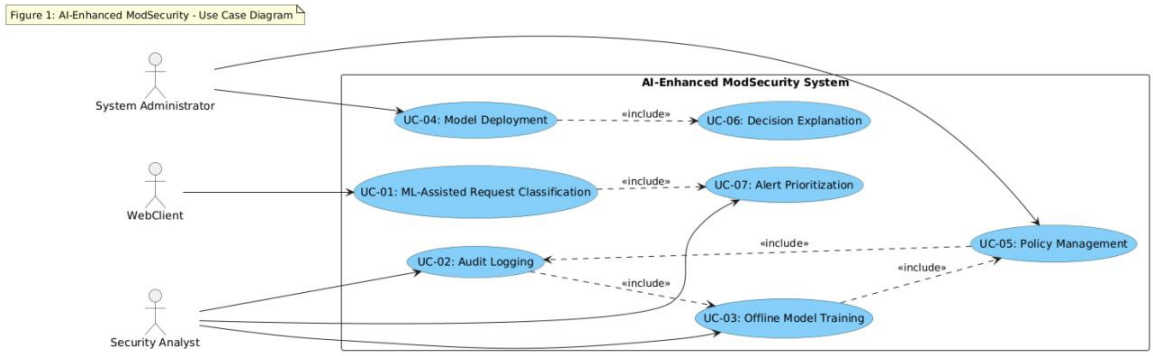
Alternative Flow:

- If explanation generation fails, the ML result is stored without explanation and the failure is logged.

3.3.3 Use Case Traceability Matrix

Use Case ID	Functional Requirement Reference
UC-01	FR-3.2.1 Machine Learning–Assisted Request Classification
UC-02	FR-3.2.2 Human-in-the-Loop Alert Prioritization
UC-03	FR-3.2.3 Offline Model Training and Evaluation
UC-04	FR-3.2.4 Safe Model Deployment and Rollback
UC-05	FR-3.2.5 Audit Logging and Traceability
UC-06	FR-3.2.6 Configurable Decision Policy Management
UC-07	FR-3.2.7 ML Decision Explanation and Justification

Figure: UML Use Case Diagram for Modintel



3.4 Non-Functional Requirements

3.4.1 Performance

- The system shall ensure that ML-assisted processing introduces no observable degradation to ModSecurity's baseline request handling, as verified through comparative testing.
- Feature extraction and inference performance shall be verifiable through controlled performance evaluation.
- Offline model training shall not affect live traffic inspection.

3.4.2 Reliability

- The system shall maintain consistent operational behavior, verified through repeated execution under identical conditions.
- Failure of auxiliary components shall not affect ModSecurity's enforcement behavior.
- Reliability shall be validated through fault-injection and recovery testing.

3.4.3 Availability

- The system shall support continuous availability of request inspection, validated through uptime monitoring.
- ML-assisted features shall be independently enableable or disableable without service interruption.

3.4.4 Security

- The system shall preserve ModSecurity's enforcement guarantees, verified through security validation testing.
- Unauthorized access to logs, models, and configurations shall be prevented through access control mechanisms.

- Security properties shall be verified through controlled attack simulations and audits.

3.4.5 Maintainability

- The system shall support modular updates, verified through component replacement testing.
- Configuration changes shall be possible without code modification.
- Maintainability shall be evaluated through documentation review and change impact analysis.

3.4.6 Portability

- The system shall be deployable across multiple environments, verified through installation and execution testing.
- No environment-specific dependencies shall be embedded in core logic.

3.5 Inverse Requirements

This section specifies what the system strictly shall not do. These constraints are critical for defining the system boundaries and ensuring safety in a security context.

- The system shall not automatically block or deny an HTTP request based solely on the Machine Learning confidence score. All blocking decisions must originate from the ModSecurity rules or be explicitly confirmed by a human administrator (Human-in-the-Loop).
- The system shall not disable or override the core ModSecurity engine or the OWASP Core Rule Set (CRS) during runtime operations. If the ML module fails, the system must fail-open (allow traffic) or fail-safe (revert to standard ModSecurity behavior) based on configuration but never crash the web server.
- The system shall not decrypt SSL/TLS traffic directly. It assumes that traffic is decrypted by the upstream web server before being inspected by ModSecurity.
- The system shall not modify the payload of the HTTP request or response. It functions strictly as a detection and alert prioritization mechanism and not as a traffic sanitization tool.
- The system shall not send sensitive log data (such as passwords, session tokens or PII) to the Machine Learning model. All data must be sanitized or hashed prior to feature extraction.
- Perform training, fine-tuning or updating of the Machine Learning model in real-time during production operation. All model training and updates shall be performed offline in a controlled, non-production environment.

- Automatically add, remove or modify any ModSecurity rules based on ML predictions or confidence scores. No automated rule learning or tuning is permitted on the system.

3.6 Design Constraints

This section outlines the limitations and standards that influence the architectural design of the project.

Software Compatibility:

- The system must be compatible with ModSecurity and the OWASP Core Rule Set (CRS).
- The integration logic must use ModSecurity's native Lua scripting interface (where applicable) or alternative connector mechanisms.

Performance/Latency:

- The ML inference process must be designed and optimized to support near real time operation suitable for web application traffic.
- The additional latency introduced by the ML check (including feature extraction and inference) must be minimized to avoid noticeable degradation of user experience.
- The final system will include performance evaluation metrics to demonstrate that the hybrid approach remains practical and acceptable under typical test conditions using commodity hardware.

Hardware Limitations:

- The system must be designed to run on standard commodity server hardware.
- It shall not require specialized hardware accelerators (such as GPUs or TPUs) for inference, ensuring it is deployable in standard academic, low-cost hosting, or small-enterprise environments.

Language Standards:

- The core integration logic within ModSecurity must be written in Lua.
- The Machine Learning service, data processing pipeline, feature extraction, model training, and inference API must be implemented in Python.

Data Availability:

- The system design is constrained by the availability of publicly labeled datasets (e.g., CSIC 2010 HTTP dataset, ECML/PKDD 2007 or other relevant web attack/benign traffic datasets).
- The feature extraction module must be designed to work with the specific fields available in these standard datasets and in ModSecurity audit logs.

3.7 Logical Database Requirements

While the system is not a data-heavy CRUD application, it requires logical storage for logs, feedback and model artifacts.

Audit Log Storage:

The system shall utilize a structured logging format (JSON) or a lightweight relational database to store audit records.

Data Integrity:

Each record must possess a unique Transaction ID that links the original ModSecurity Alert to the ML Inference Result.

Analyst Feedback Loop:

A logical storage entity must exist to store "Ground Truth" labels provided by the Security Analyst.

Fields Required:

Transaction_ID, Original_Alert_Type, Analyst_Label (False Positive/True Positive), Timestamp.

Data Retention:

The system must support a retention policy where logs older than a configurable period are archived or deleted to prevent storage exhaustion on the host server.

Model Artifact Storage:

The system requires a file-system-based storage area for versioned ML model artifacts

(.pkl, .json, or .model files) to allow for rollback to previous model versions if a new deployment performs poorly.

3.8 Other Requirements

3.8.1 Training-related Requirements

Users (Security Analysts/Admins) must possess a basic understanding of Web Application Firewalls and HTTP status codes to interpret the dashboard correctly.

User Manual must be provided that explains how to interpret the "Confidence Score" and how to use the dashboard to label a request as a False Positive for future retraining.

3.8.2 Packaging Requirements

Distribution:

The software shall be delivered as a version-controlled source code repository (GitHub).

Containerization:

For ease of testing and defense demonstration, the system should ideally be packaged with a Docker Compose file that orchestrates the Web Server (with ModSecurity), the ML API service and the Dashboard.

Documentation:

The package must include:

A README.md file containing installation steps.

A requirements.txt (or equivalent) for Python dependencies.

A setup script to initialize the environment.

3.8.3 Legal and Compliance Requirements

Licensing:

The project relies on open-source software. The final deliverable must respect the Apache License v2.0 (ModSecurity) and the licensing terms of the utilized datasets.

Privacy:

To comply with general privacy ethics, the logging mechanism must be configured to mask

or exclude specific HTTP headers (like Authorization or Cookie) that may contain sensitive user credentials.

4. Change Management Process

This section describes how changes to the requirements and scope will be managed throughout the project lifecycle, adhering to the Scrum methodology outlined in the proposal.

4.1 Change Identification

Changes to the SRS may be identified by:

Project Team Members: Upon discovering technical limitations during the Implementation Sprints.

Project Advisor: During bi-weekly feedback sessions or sprint reviews.

External Factors: Unavailability of specific datasets or tools.

4.2 Change Submission

All proposed changes must be documented in the Project Backlog.

Significant changes require a formal Change Request submitted to the group leader and the advisor.

4.3 Change Approval

Minor Changes: Can be approved by the internal team during Sprint Planning meetings if they do not impact the project deadline.

Major Changes: Changes affecting the scope, objectives or final deliverables require approval from the Project Advisor.

Once approved, the SRS document version number will be incremented, and the change will be recorded in the document history log.

4.4 Documentation Update

The project documentation lead is responsible for updating the SRS and communicating the changes to the development team through Telegram or Email.

REFERENCES

- OWASP ModSecurity Core Rule Set (CRS) v4.x Documentation

  https://coreruleset.org/

- ModSecurity v3 Reference Manual

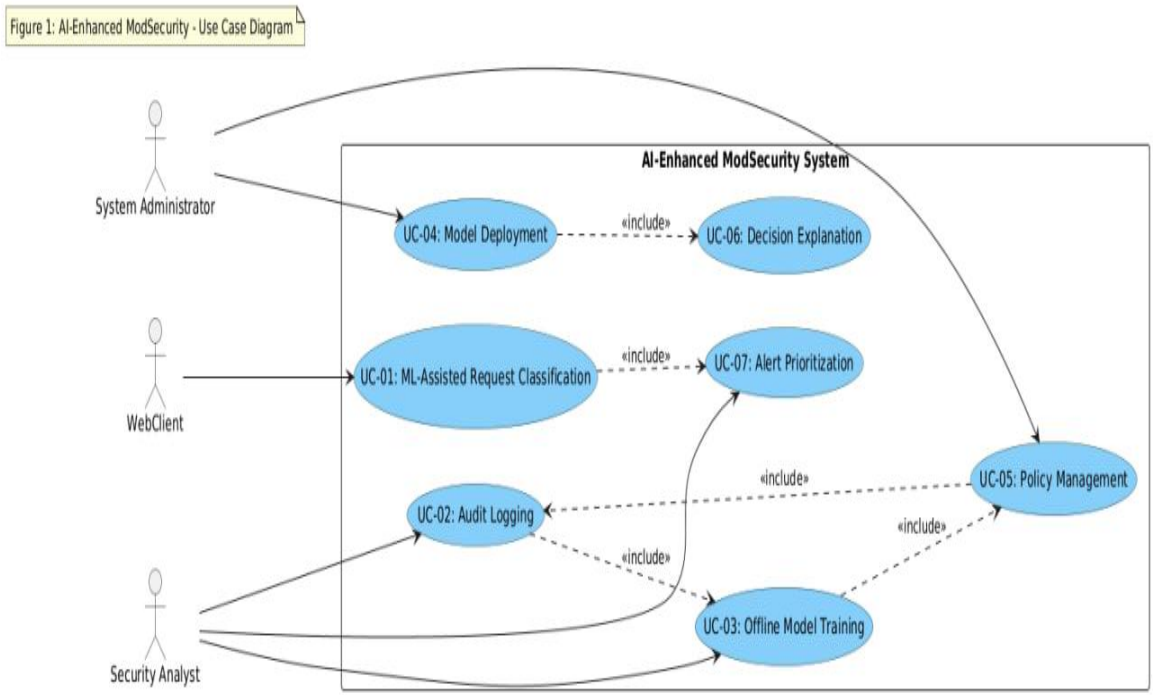
  https://github.com/SpiderLabs/ModSecurity/wiki/Reference-Manual

- scikit-learn Machine Learning Library Documentation

  https://scikit-learn.org/stable/

- CSIC 2010 HTTP Dataset (for web attack detection research)

- Relevant academic papers on ML-based false positive reduction in WAFs

APPENDICES

Appendix 1